

SMART FIRE BOX

IoT-Based Intelligent Fire Detection & Suppression System

Project Report

13. Project Photographs

Zone 1 — Front View with Sensors and Emergency Door

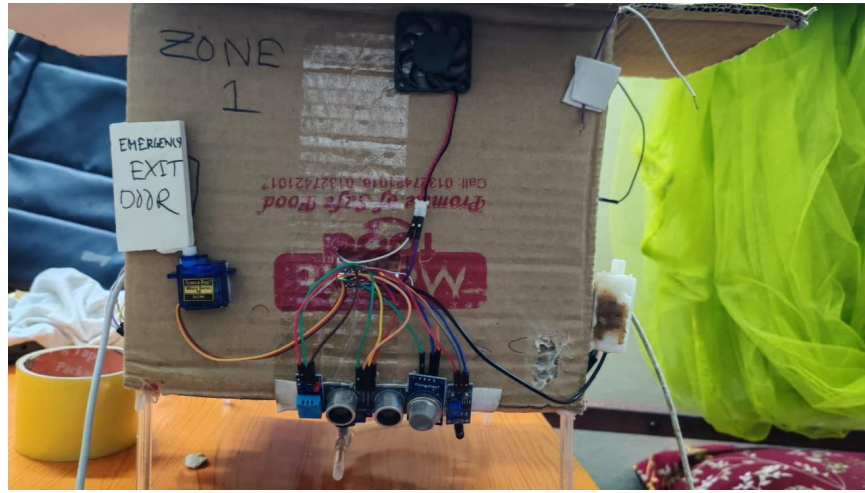


Figure 3: Zone 1 front panel — emergency exit door servo, HC-SR04, flame sensor, MQ-2, and exhaust fan

Internal Wiring — ESP32 Boards and Breadboard Assembly



Figure 4: Top-down view — two ESP32 DevKit V1 boards on breadboards with relay module and wiring

Sensor Cluster and Nozzle Servo — Bottom View

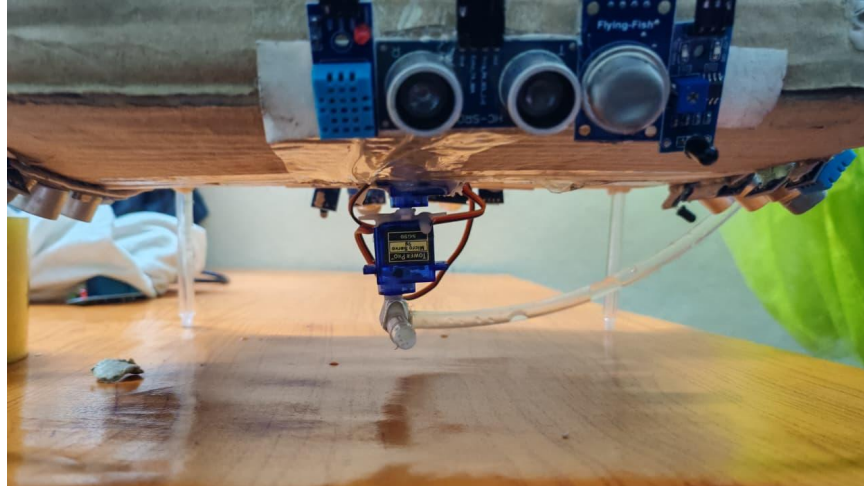


Figure 5: Underside view — DHT11, HC-SR04, flame sensor, and SG90 nozzle servo mechanism

Zone 2 — Side View with Pump and Wiring

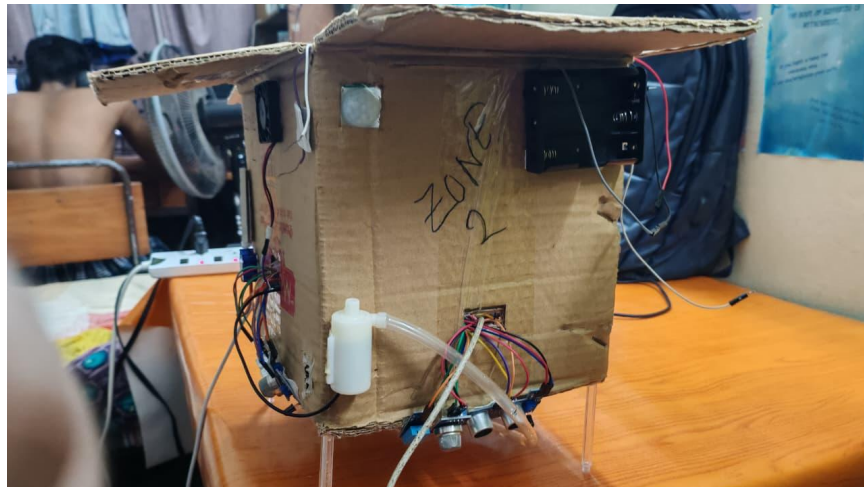


Figure 6: Side view of Zone 2 — water pump container, sensor wiring, and exhaust fan

Blynk Interface:

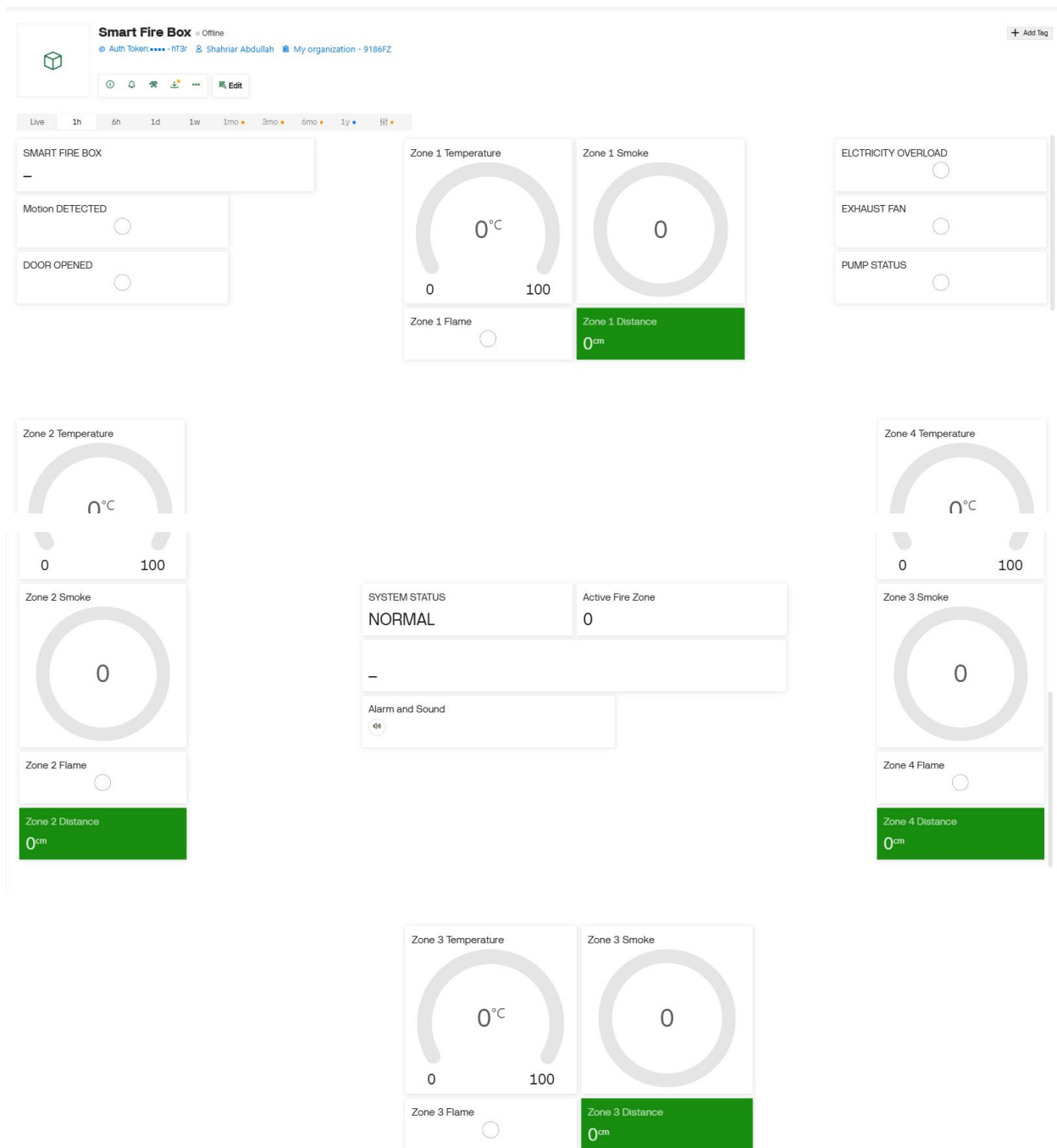


Figure 7: Blynk Interface

14. Source Code

14.1 ESP32 #1 — Sensor Unit (sensor_sender_esp32.ino)

```
#include <DHT.h>

#define TX2_PIN    15
#define RX2_PIN    12
#define BAUD_RATE  9600
#define NUM_ZONES  4
#define DHT_TYPE    DHT11
#define SONAR_TIMEOUT_US  30000
#define SOUND_SPEED_CM_US 0.0343
#define SEND_INTERVAL_MS 1000

const uint8_t FLAME_PINS[NUM_ZONES] = {13, 14, 18, 23};
const uint8_t SMOKE_PINS[NUM_ZONES] = {34, 36, 32, 33};
const uint8_t DHT_PINS[NUM_ZONES]   = {4, 16, 19, 26};
const uint8_t SONAR_TRIG[NUM_ZONES] = {5, 17, 21, 27};
const uint8_t SONAR_ECHO[NUM_ZONES] = {35, 39, 22, 25};

DHT dhtSensors[NUM_ZONES] = {
    DHT(DHT_PINS[0], DHT_TYPE),
    DHT(DHT_PINS[1], DHT_TYPE),
    DHT(DHT_PINS[2], DHT_TYPE),
    DHT(DHT_PINS[3], DHT_TYPE)
};

struct ZoneData {
    int    flame;
    int    smoke;
    float  temperature;
    int    distance;
};

ZoneData zones[NUM_ZONES];
unsigned long lastSendTime = 0;

void setup() {
    Serial2.begin(BAUD_RATE, SERIAL_8N1, RX2_PIN, TX2_PIN);
    for (int i = 0; i < NUM_ZONES; i++) {
        pinMode(FLAME_PINS[i], INPUT);
        dhtSensors[i].begin();
        pinMode(SONAR_TRIG[i], OUTPUT);
        pinMode(SONAR_ECHO[i], INPUT);
        digitalWrite(SONAR_TRIG[i], LOW);
    }
    delay(2000);
}

void loop() {
    if (millis() - lastSendTime >= SEND_INTERVAL_MS) {
        lastSendTime = millis();
        readAllSensors();
        sendToReceiver();
    }
}

void readAllSensors() {
    for (int i = 0; i < NUM_ZONES; i++) {
        zones[i].flame = (digitalRead(FLAME_PINS[i]) == LOW) ? 1 : 0;
        zones[i].smoke = analogRead(SMOKE_PINS[i]);
        float t = dhtSensors[i].readTemperature();
        zones[i].temperature = isnan(t) ? -999.0 : t;
    }
}
```

```

        zones[i].distance = (int)readSonar(i);
    }
}

float readSonar(int idx) {
    digitalWrite(SONAR_TRIG[idx], LOW);
    delayMicroseconds(2);
    digitalWrite(SONAR_TRIG[idx], HIGH);
    delayMicroseconds(10);
    digitalWrite(SONAR_TRIG[idx], LOW);
    long duration = pulseIn(SONAR_ECHO[idx], HIGH, SONAR_TIMEOUT_US);
    if (duration == 0) return -1.0;
    return (duration * SOUND_SPEED_CM_US) / 2.0;
}

void sendToReceiver() {
    String packet = "";
    for (int i = 0; i < NUM_ZONES; i++) {
        if (i > 0) packet += ";";
        packet += "Z" + String(i + 1);
        packet += "," + String(zones[i].flame);
        packet += "," + String(zones[i].smoke);
        packet += "," + String(zones[i].temperature, 1);
        packet += "," + String(zones[i].distance);
    }
    packet += "#";
    Serial2.print(packet);
}

```

14.2 ESP32 #2 — Actuator Unit (actuator.ino)

```

#define BLYNK_TEMPLATE_ID "TMPL6_cwACJRd"
#define BLYNK_TEMPLATE_NAME "Smart Fire Box"
#define BLYNK_AUTH_TOKEN "f4BI39vfC6WkN08KEkai2V9IoGcXhT3r"

#include <WiFi.h>
#include <WiFiClient.h>
#include <BlynkSimpleEsp32.h>
#include <ESP32Servo.h>

char ssid[] = "Protooy";
char pass[] = "987654321";
BlynkTimer timer;

#define RX_PIN 16
#define TX_PIN 17
#define BOTTOM_SERVO_PIN 18
#define TOP_SERVO_PIN 19
#define FAN_RELAY_PIN 21
#define PUMP_RELAY_PIN 22
#define PIR_SENSOR_PIN 13
#define CURRENT_SENSOR_PIN 34
#define BUZZER_PIN 32
#define DOOR_SERVO_PIN 33

Servo bottomServo;
Servo topServo;
Servo doorServo;

struct ZoneData {
    int flame;
    int smoke;
    float temperature;
}

```

```

    int distance;
};

ZoneData zones[4] = {{0,0,0.0,0},{0,0,0.0,0},{0,0,0.0,0},{0,0,0.0,0}};

enum SystemState {
    STATE_NORMAL, STATE_HIGH_TEMP, STATE_SMOKE, STATE_FIRE, STATE_OVER_CURRENT
};

char rxBuffer[256];
int rxIndex = 0;
bool packetReady = false;
SystemState currentState = STATE_NORMAL;
int activeFireZone = -1;
int currentActiveZone = -1;
bool isDoorOpen = false;
bool isFanOn = false;
bool isPumpOn = false;
bool isBuzzerOn = false;
int currentAdcGlobal = 0;
String systemStatusStr = "NORMAL";
String alarmMessage = "";
bool isMotionDetected = false;

TaskHandle_t blynkTask;

void blynkTaskCode(void* parameter) {
    for (;;) {
        if (WiFi.status() == WL_CONNECTED) {
            if (!Blynk.connected()) Blynk.connect();
            else Blynk.run();
        }
        timer.run();
        delay(10);
    }
}

void setup() {
    Serial.begin(115200);
    Serial2.begin(9600, SERIAL_8N1, RX_PIN, TX_PIN);
    pinMode(FAN_RELAY_PIN, OUTPUT); digitalWrite(FAN_RELAY_PIN, HIGH);
    pinMode(PUMP_RELAY_PIN, OUTPUT); digitalWrite(PUMP_RELAY_PIN, HIGH);
    pinMode(BUZZER_PIN, OUTPUT); digitalWrite(BUZZER_PIN, LOW);
    pinMode(CURRENT_SENSOR_PIN, INPUT);
    pinMode(PIR_SENSOR_PIN, INPUT);
    bottomServo.attach(BOTTOM_SERVO_PIN);
    topServo.attach(TOP_SERVO_PIN);
    doorServo.attach(DOOR_SERVO_PIN);
    currentActiveZone = -1;
    isDoorOpen = false;
    doorServo.write(0); bottomServo.write(0); topServo.write(0);
    WiFi.mode(WIFI_STA);
    WiFi.begin(ssid, pass);
    Blynk.config(BLYNK_AUTH_TOKEN);
    timer.setInterval(1000L, sendBlynkData);
    xTaskCreatePinnedToCore(blynkTaskCode, "BlynkTask", 10000, NULL, 1, &blynkTask, 0);
}

void loop() {
    receivePacket();
    if (packetReady) parsePacket();
    isMotionDetected = digitalRead(PIR_SENSOR_PIN) == HIGH;
    processSystem();
    delay(50);
}

```

```

}

void receivePacket() {
    while (Serial2.available() > 0) {
        if (packetReady) break;
        char c = Serial2.read();
        if (c == '\r' || c == '\n') continue;
        if (c == '#') {
            rxBuffer[rxIndex] = '\0';
            packetReady = true;
            rxIndex = 0;
        } else {
            if (rxIndex < sizeof(rxBuffer) - 1) rxBuffer[rxIndex++] = c;
        }
    }
}

void parsePacket() {
    char* token = strtok(rxBuffer, ";");
    while (token != NULL) {
        int zoneNum, flame, smoke, dist; float temp;
        if (sscanf(token, "%d,%d,%d,%f,%d", &zoneNum, &flame, &smoke, &temp, &dist) == 5) {
            if (zoneNum >= 1 && zoneNum <= 4) {
                int index = zoneNum - 1;
                zones[index].flame = flame;
                zones[index].smoke = smoke;
                zones[index].temperature = temp;
                zones[index].distance = dist;
            }
        }
        token = strtok(NULL, ";");
    }
    packetReady = false;
}

void processSystem() {
    long sum = 0;
    for(int i = 0; i < 20; i++) sum += analogRead(CURRENT_SENSOR_PIN);
    int currentAdc = sum / 20;
    currentAdcGlobal = currentAdc;
    bool overCurrent = (currentAdc > 2600);
    activeFireZone = -1;
    bool hasSmoke = false, hasHighTemp = false;
    for (int i = 0; i < 4; i++) {
        if (zones[i].flame == 1) { activeFireZone = i + 1; break; }
        if (zones[i].smoke > 1000) hasSmoke = true;
        if (zones[i].temperature > 35.0) hasHighTemp = true;
    }
    if (overCurrent) currentState = STATE_OVER_CURRENT;
    else if (activeFireZone != -1) currentState = STATE_FIRE;
    else if (hasSmoke) currentState = STATE_SMOKE;
    else if (hasHighTemp) currentState = STATE_HIGH_TEMP;
    else currentState = STATE_NORMAL;

    switch (currentState) {
        case STATE_NORMAL: systemStatusStr="NORMAL"; alarmMessage=""; break;
        case STATE_HIGH_TEMP: systemStatusStr="HIGH TEMP"; alarmMessage="High Temp!"; break;
        case STATE_SMOKE: systemStatusStr="SMOKE"; alarmMessage="Smoke Detected!"; break;
        case STATE_FIRE: systemStatusStr="FIRE"; alarmMessage="Fire Detected!"; break;
        case STATE_OVER_CURRENT: systemStatusStr="OVER CURRENT"; alarmMessage="Overcurrent!"; break;
    }

    if (currentState==STATE_OVER_CURRENT||currentState==STATE_FIRE) {
        digitalWrite(BUZZER_PIN, HIGH); isBuzzerOn = true;
    }
}

```



```

    } else { digitalWrite(BUZZER_PIN, LOW); isBuzzerOn = false; }

    if (hasSmoke) { digitalWrite(FAN_RELAY_PIN, LOW); isFanOn = true; }
    else { digitalWrite(FAN_RELAY_PIN, HIGH); isFanOn = false; }

    if (activeFireZone != -1) {
        bool zoneChanged = (currentActiveZone != activeFireZone);
        moveToZone(activeFireZone);
        if (isMotionDetected) openDoor(); else closeDoor();
        if (zoneChanged) delay(1500);
        pumpOn();
    } else { closeDoor(); pumpOff(); }
}

void moveToZone(int zone) {
    if (currentActiveZone == zone) return;
    currentActiveZone = zone;
    int bA=0, tA=0;
    switch(zone) {
        case 1: bA=0; tA=0; break;
        case 4: bA=90; tA=0; break;
        case 3: bA=180; tA=0; break;
        case 2: bA=180; tA=90; break;
    }
    bottomServo.write(bA); topServo.write(tA);
}

bool openDoor() {
    if (!isDoorOpen) { doorServo.write(90); isDoorOpen=true; return true; }
    return false;
}

bool closeDoor() {
    if (isDoorOpen) { doorServo.write(0); isDoorOpen=false; return true; }
    return false;
}

void pumpOn() { digitalWrite(PUMP_RELAY_PIN, LOW); isPumpOn = true; }
void pumpOff() { digitalWrite(PUMP_RELAY_PIN, HIGH); isPumpOn = false; }

void sendBlynkData() {
    Blynk.virtualWrite(V0,zones[0].temperature);
    Blynk.virtualWrite(V1,zones[0].smoke);
    Blynk.virtualWrite(V2,zones[0].flame);
    Blynk.virtualWrite(V3,zones[0].distance);
    Blynk.virtualWrite(V4,zones[1].temperature);
    Blynk.virtualWrite(V5,zones[1].smoke);
    Blynk.virtualWrite(V6,zones[1].flame);
    Blynk.virtualWrite(V7,zones[1].distance);
    Blynk.virtualWrite(V8,zones[2].temperature);
    Blynk.virtualWrite(V9,zones[2].smoke);
    Blynk.virtualWrite(V10,zones[2].flame);
    Blynk.virtualWrite(V11,zones[2].distance);
    Blynk.virtualWrite(V12,zones[3].temperature);
    Blynk.virtualWrite(V13,zones[3].smoke);
    Blynk.virtualWrite(V14,zones[3].flame);
    Blynk.virtualWrite(V15,zones[3].distance);
    Blynk.virtualWrite(V16,isPumpOn?1:0);
    Blynk.virtualWrite(V17,isMotionDetected?1:0);
    Blynk.virtualWrite(V18,currentState==STATE_OVER_CURRENT?1:0);
    int angle=0;
    if(currentActiveZone==1) angle=0;
    else if(currentActiveZone==4) angle=90;
    else if(currentActiveZone==3) angle=180;
    else if(currentActiveZone==2) angle=180;
}

```

```
Blynk.virtualWrite(V19,angle);
Blynk.virtualWrite(V20,isFanOn?1:0);
Blynk.virtualWrite(V21,isDoorOpen?1:0);
Blynk.virtualWrite(V22,systemStatusStr);
Blynk.virtualWrite(V23,activeFireZone== -1?0:activeFireZone);
Blynk.virtualWrite(V24,alarmMessage);
Blynk.virtualWrite(V25,isBuzzerOn?1:0);
}
```